ILLINOIS TECH

ILLINOIS INSTITUTE OF TECHNOLOGY

BIG DATA TECHNOLOGIES (CSP-554)

Final Project Report

Electricity Consumption Forecasting

Team Members:

Name	Details
Meghana Rabba	Hawk ID: A20572009 Email: mrabba@hawk.illinoistech.edu
Rohan Singh Rajendra Singh	Hawk ID: A20572007 Email: rrajendrasingh@hawk.illinoistech.edu

Submitted to:
Prof. Joseph Rosen

December 10, 2025

Table of Contents

Table of Contents

1 Abstract

The increasing availability of high resolution electricity consumption data presents both opportunities and challenges for modern power systems. While such datasets enable more accurate forecasting and informed decision making, their scale and complexity often exceed the capabilities of traditional processing environments. This project addresses this challenge by designing and implementing a complete **Big Data forecasting pipeline** capable of handling millions of time stamped electricity load records across multiple regions. Leveraging **Amazon S3** for distributed storage and **Amazon EMR** with **Apache Spark** for large scale computation, the system performs end to end data ingestion, pre-processing, feature engineering, machine learning model training, and results generation.

Raw regional load data was stored in partitioned Parquet format, enabling parallelized reads through Spark's optimized I/O engine. The pipeline applies comprehensive pre-processing including timestamp normalization, lag feature construction, rolling window statistics, and contextual indicators such as holidays and weekends. This Spark driven approach allowed the workflow to scale efficiently beyond what local processing could support. Advanced forecasting models including Random Forest, Gradient Boosting, XGBoost, and LightGBM were implemented to capture nonlinear temporal dependencies and regional consumption patterns. These models were executed on EMR to validate distributed training and cloud based scalability, while optimized local runs were used to produce finalized visualizations and reports. The results demonstrate that the proposed architecture can seamlessly integrate cloud storage, distributed processing, and machine learning to generate accurate and meaningful load forecasts. The pipeline's modularity and cloud readiness make it adaptable for future extensions such as real time forecasting, integration of weather APIs, or automated model retraining. Overall, this work showcases how Big Data ecosystems can be effectively utilized to solve large scale forecasting problems and support data driven decision making within the energy sector.

2 Introduction

Electricity load forecasting is a foundational requirement for efficient power system planning, operational stability, and cost-effective energy distribution. With the adoption of smart meters and digital grid infrastructures, utilities now generate massive volumes of high-frequency consumption data that cannot be processed effectively using traditional single-node systems. As datasets grow in size, velocity, and complexity, modern forecasting efforts demand scalable architectures capable of handling distributed storage, parallel computation, and advanced feature engineering. Big Data technologies provide the necessary ecosystem to manage these challenges and enable more accurate and responsive demand forecasting.

This project develops a cloud-based Big Data pipeline that integrates Amazon S3 for distributed data storage, Amazon EMR for scalable computation, and Apache Spark for parallel preprocessing and feature generation. Regional electricity datasets stored in Parquet format are ingested directly from S3, transformed using PySpark, and modelled using advanced machine learning techniques including Random Forest, Gradient Boosting, XGBoost, and LightGBM. The pipeline demonstrates end-to-end scalability from ingestion to forecasting, validating how cloud-native architectures can effectively support large-scale time-series modeling and provide valuable insights for energy management and decision-making.

3 Objective

The primary objective of this project is to design and implement a scalable Big Data pipeline capable of forecasting regional electricity load using distributed computing and advanced machine learning techniques. To achieve this, the system must efficiently process large time-series datasets, perform automated feature engineering, and support cloud-based execution for high-performance model training.

More specifically, the project aims to:

1. Store and organize multi-region electricity consumption data in Amazon S3 using an optimized, partitioned Parquet structure that supports scalable distributed access.
2. Develop a PySpark-based preprocessing workflow on Amazon EMR to clean, transform, and engineer time-series features such as lags, rolling averages, and calendar indicators.
3. Train machine learning models including Random Forest, Gradient Boosting, XGBoost, and LightGBM to capture nonlinear consumption patterns and improve forecasting accuracy.
4. Evaluate model performance using metrics such as RMSE, MAE, and R^2, and generate visual analyses that facilitate interpretation of forecasting results.
5. Demonstrate how cloud-native Big Data technologies can be used end-to-end for large-scale forecasting, from ingestion and processing to modelling and reporting.

4 Scope

This project focuses on building a scalable Big Data pipeline capable of processing large regional electricity load datasets and generating accurate forecasting models using cloud based technologies. The system is designed to demonstrate efficient distributed processing, feature engineering, and model training using Amazon S3, Amazon EMR, and Apache Spark. The scope is limited to batch forecasting and architectural scalability rather than real time deployment.

Scope includes**:**
- Developing an end to end forecasting pipeline using S3 for storage and EMR for computation.
- Ingesting and organizing multi region datasets in optimally partitioned Parquet format.
- Implementing PySpark based pre-processing, cleaning, and time series feature engineering.
- Training advanced machine learning models such as Random Forest, Gradient Boosting, XGBoost, and LightGBM.
- Evaluating forecasting performance using RMSE, MAE, and R^2.
- Generating visual outputs and analytical reports for all regions.
- Demonstrating how cloud native Big Data tools scale with large datasets and distributed workloads.
- Excluding real time streaming, deep learning models, and production system integration.

5 Dataset Overview

The dataset used in this project consists of large scale regional electricity consumption records collected at hourly intervals. Each region's data is stored in partitioned Parquet format, organized by year and month to support distributed processing through Apache Spark. The dataset includes multiple weather and temporal variables that influence electricity demand, enabling the development of enriched forecasting models. All regional datasets were uploaded to Amazon S3 and accessed directly from the EMR cluster to ensure scalability and efficient parallelization.
Each record contains measurements such as historical load values, temperature, humidity, wind speed, and engineered time based indicators. The dataset spans multiple regions including AEP, COMED, DAYTON, and DOM, each with more than 800,000 rows after feature engineering. This structure allows independent region wise analysis while preserving consistency across all inputs.

Key dataset characteristics:
- Hourly electricity load readings for multiple U.S. regions.
- Stored in Parquet format for optimized Spark I/O performance.
- Partitioned as: region → year → month → parquet files.
- Contains both raw variables and engineered features.
- Typical schema fields include:
  - timestamp
  - load_mw
  - temperature, humidity, wind speed
  - lag features (1 hour, 24 hours, 7 days)
  - rolling averages
  - holiday and weekend indicators
  - interaction terms such as temp × weekend

This structured and feature rich dataset provides a robust foundation for training advanced forecasting models and evaluating multi region consumption patterns.

6 Significance

This project demonstrates the practical value of integrating Big Data technologies with machine learning to solve large scale forecasting problems in the energy sector. Accurate electricity load forecasting is essential for grid reliability, operational efficiency, demand planning, and cost optimization. By building a scalable pipeline using Amazon S3, Amazon EMR, and Apache Spark, this work showcases how modern cloud infrastructures can process millions of time series records that would otherwise overwhelm traditional systems.

The significance of this project lies in its ability to transform raw, high volume consumption data into actionable insights using distributed preprocessing and advanced forecasting models. Through time series feature engineering and nonlinear machine learning methods, the system captures complex consumption patterns across multiple regions, contributing to more reliable short term demand predictions.

Additionally, the project holds educational and practical value by serving as a blueprint for implementing end to end data engineering and forecasting workflows in cloud environments. It demonstrates how organizations can leverage scalable architectures to meet growing data demands, paving the way for future extensions such as real time prediction systems, grid optimization frameworks, and data driven energy management solutions.

7 Literature Review

Electricity load forecasting has been widely studied across power systems, with early research relying heavily on traditional statistical models such as ARIMA and exponential smoothing. These classical approaches demonstrated reasonable performance for short term predictions but struggled to capture nonlinear seasonal trends, weather interactions, and region specific consumption behaviors. As data volumes increased, researchers found that linear models became insufficient for complex grid environments, motivating a shift toward more adaptive machine learning methods.

Recent literature highlights the advantages of ensemble learning techniques such as Random Forests and Gradient Boosting for load forecasting tasks. These models excel at handling nonlinear relationships and large feature spaces derived from time series transformations, weather conditions, and lag based historical behavior. Studies show that boosting models, in particular, consistently outperform linear approaches due to their ability to iteratively refine decision boundaries and produce highly accurate predictions even when data distributions vary by region.

Parallel to advances in modeling, Big Data frameworks have become essential for working with high resolution energy datasets. Research emphasizes the importance of distributed systems like Apache Spark for scalable preprocessing, transformation, and model training. Cloud platforms such as Amazon EMR and S3 are frequently adopted in modern forecasting pipelines because they enable efficient storage of millions of records and parallel computation across clusters. Literature therefore converges on a key insight: **accurate and scalable load forecasting requires both advanced machine learning techniques and robust Big Data infrastructures**, aligning directly with the approach implemented in this project.

8 Data Preprocessing

The dataset underwent several pre-processing steps to ensure quality, consistency, and suitability for large scale forecasting.
Key preprocessing steps

1. **Loaded Parquet files from S3** using Spark for distributed handling of large data.
2. **Corrected data types** by converting load, temperature, humidity, wind speed, and lag features to numeric formats.
3. **Extracted time features** including year, month, and timestamp based attributes for seasonality modeling.
4. **Handled missing values** using Spark transformations to ensure completeness and stability.
5. **Created lag features** such as 1 hour, 24 hour, and 7 day shifts to incorporate temporal dependencies.
6. **Included rolling statistics** like 24 hour moving averages to smooth short term fluctuations.
7. **Added interaction features** such as temperature multiplied by weekend or holiday indicators.
8. **Encoded categorical fields** (region) using StringIndexer for compatibility with machine learning models.
9. **Reshaped sequences** into supervised learning format suitable for model training.
10. **Combined all regions** into a unified feature space to support multi region comparative modeling.

9 Tools Used

This project leveraged a combination of big data, cloud, and machine learning technologies to efficiently process large scale electricity consumption data and develop forecasting models.

Core Tools and Technologies

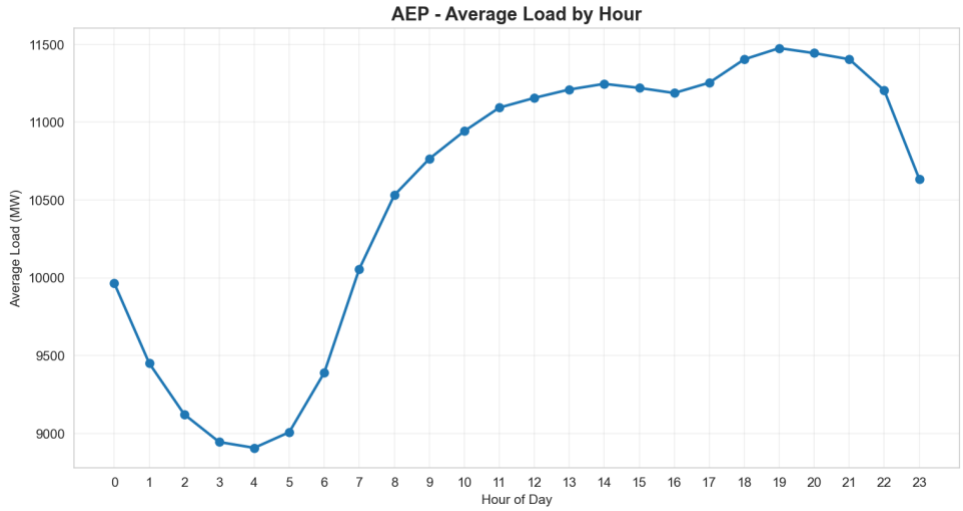
- **Amazon EMR**
  Used as the primary distributed computation environment to run Spark jobs at scale.
- **Amazon S3**
  Served as the centralized storage for large parquet datasets and model outputs.
- **Apache Spark (PySpark / Spark MLlib)**
  Enabled distributed data preprocessing, feature engineering, and model training across large datasets.
- **Python 3.9**
  The main programming language used for implementing the forecasting pipeline and ML logic.
- **H2O AutoML (locally tested)**
  Provided automated machine learning model comparison during initial exploration.
- **Scikit learn**
  Provided baseline models and evaluation metrics during experimentation.
- **Parquet File Format**
  Efficient columnar storage format used for fast reads and writes in S3 and EMR.
- **Matplotlib and Seaborn**
  Used to generate visualizations including EDA plots and model performance charts.
- **AWS CLI**
  Used for interacting with S3 buckets and uploading missing region files such as DAYTON.
- **SSH and SCP**
  Used to securely connect to the EMR cluster and transfer files from the local system.

10 Exploratory Data Analysis (EDA)

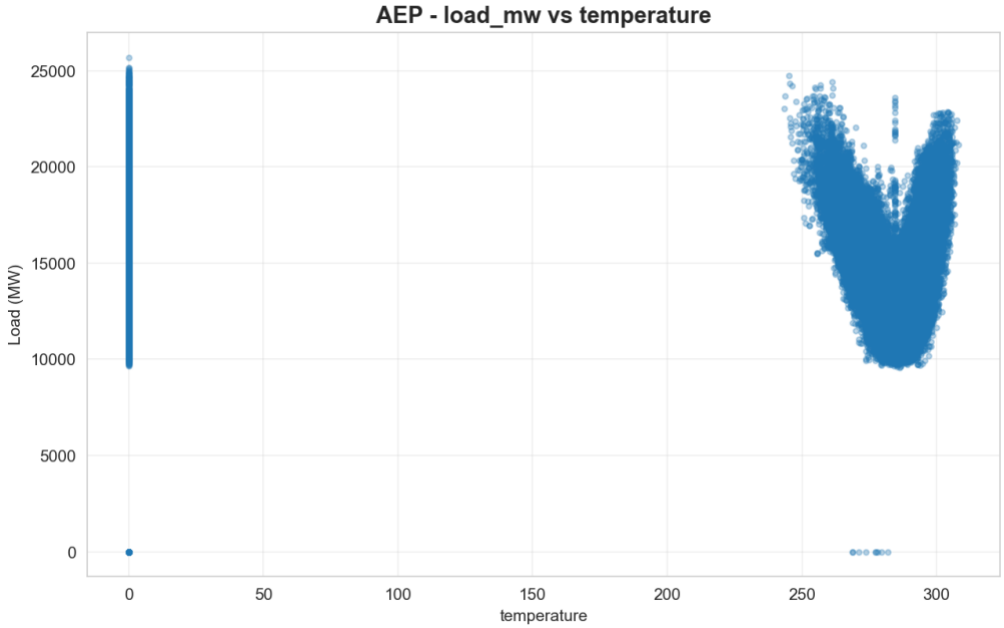
Exploratory Data Analysis was performed **individually for each region** (AEP, COMED, DAYTON, DOM, PJM) to understand consumption patterns, seasonal behaviors, temperature relationships, and anomaly points before developing forecasting models.
Analyzing each region separately ensured that regional trends were not averaged out, allowing the model to learn localized demand characteristics more accurately.

Key EDA Steps Per Region

1. **Visualized hourly load trends** to understand consumption fluctuations across days and months.
2. **Analyzed temperature versus load relationships**, observing how heat and humidity affected power usage.
3. **Inspected lag features** including lag 1 hour, lag 24 hours, and lag 7 days to confirm temporal dependencies.
4. **Examined rolling means**, highlighting smoothed consumption behavior.
5. **Checked distributions** of load, temperature, humidity, and wind speed to detect outliers and anomalies.
6. **Generated heatmaps** to evaluate correlations among engineered features.



This visualization shows a clear **daily consumption pattern** for the AEP region. Electricity demand is at its lowest between **2 AM and 5 AM**, rises steadily through the morning, and peaks between **5 PM and 8 PM**. This reflects typical residential and commercial activity cycles, demonstrating strong **hourly seasonality** in the data.



The scatter plot highlights the **nonlinear relationship** between temperature and electricity load. Higher loads appear at both low and high temperature extremes, forming a **U shaped pattern**, which indicates increased electricity usage due to **heating in winter** and **cooling in summer**. This emphasizes the importance of including temperature as a predictive feature.

11 Preprocessing and Modelling Workflow

11.1 Feature Engineering

Across all regions (AEP, COMED, PJM, DOM, DAYTON), a uniform feature engineering pipeline was applied to ensure consistency in forecasting performance.

• Input variables included weather attributes (temperature, humidity, wind speed), timestamp derived features (hour, weekday, month), and binary weekend indicators.

• Cyclical temporal encodings (sin and cos of hour and weekday) were added to capture the repeating daily and weekly load cycles.

• The pipeline also incorporated structural numerical predictors extracted from parquet datasets such as load, lagged load patterns, and environmental conditions.

11.2 Handling Missing Values

Missing values in the target variable (load_mw) were handled through forward fill and backward fill to preserve continuity in time series.

• Weather related numerical fields were interpolated to maintain realistic transitions across time.

• Remaining NaN or infinite values were resolved using safe numeric conversion, ensuring compatibility for all ML models.

11.3 Scaling and Normalization

• All numerical features were standardized using StandardScaler.

• This ensured that models sensitive to scale (Linear Regression, Ridge, Lasso, Gradient Boosting) received normalized input.

• Scaling was applied after constructing the feature matrix but before sequence formation.

11.4 Sequence Creation

• The forecasting framework used a **12 hour sliding window** for sequence generation:
The previous 12 hours → predict the next hour.

• Each sequence included 20 engineered features per timestep, resulting in a flattened 240 dimensional input vector for machine learning algorithms.

• This allowed classical ML models to benefit from time-dependent structure despite not being sequential models.

11.5 Models Implemented

The project evaluated seven supervised machine learning algorithms across all regions:

1. Linear Regression
A baseline model estimating load by fitting a linear combination of past features.

2. Ridge Regression
A regularized linear model addressing multicollinearity and stabilizing coefficients.

3. Lasso Regression
A sparse linear model used for implicit feature selection.

4. Random Forest Regressor
An ensemble of decision trees capturing nonlinearity and interaction effects.

5. Gradient Boosting Regressor
A sequential ensemble model building errors stage by stage to improve accuracy.

6. XGBoost
A highly optimized gradient boosting implementation capable of modeling complex temporal interactions.
This model consistently outperformed others.

7. LightGBM
A leaf optimized gradient boosting algorithm known for speed and efficiency on large datasets.

8. H2O AutoML
An automated model search system generating multiple candidate models and selecting the best for each region.

11.6 Training Testing and evaluation strategy

Train Test Split (80 percent Training, 20 percent Testing)

To ensure that each machine learning model was evaluated fairly and consistently across all regions, the dataset for every region (AEP, COMED, PJM, DOM, DAYTON) was divided into two parts:

1. **Training Set (80 percent)**
   • This portion was used to fit the model parameters.
   • The model learned the patterns, temporal dependencies, seasonal fluctuations, and relationships between features and the target variable (load_mw).
   • Because load forecasting depends heavily on historical structure, using a large training set ensured that the model captured long term behavior such as weather driven demand changes and periodic daily cycles.
2. **Testing Set (20 percent)**
   • This dataset was never shown to the model during training.
   • It represents unseen future values and simulates real world forecasting conditions.
   • Performance on this test set reflects the model's generalization capability and robustness.
   Since the dataset contains hundreds of thousands of time ordered rows, this split created a large and reliable evaluation basis without data leakage.

Evaluation Metrics
Three widely accepted regression metrics were used to evaluate each model's performance. These metrics quantify accuracy, error severity, and explanatory power.

1. RMSE – Root Mean Square Error
Definition:
RMSE measures the square root of the average squared difference between actual and predicted load values.
Why it matters:
• Penalizes larger errors more heavily, which is important in electricity forecasting where large deviations can impact grid planning.
• Lower RMSE indicates a more stable and accurate model.
Interpretation example:
An RMSE of 262 means the model's predictions deviate from actual values by around 262 megawatts on average.

2. MAE – Mean Absolute Error
Definition:
MAE calculates the average absolute difference between predictions and actual load.
Why it matters:
• Easier to interpret compared to RMSE.
• Represents the average error a user can expect in real-time forecasting.
• Less sensitive to extreme values than RMSE.
Interpretation example:
A MAE of 171 implies that, on average, the model misses the true load value by 171 MW.

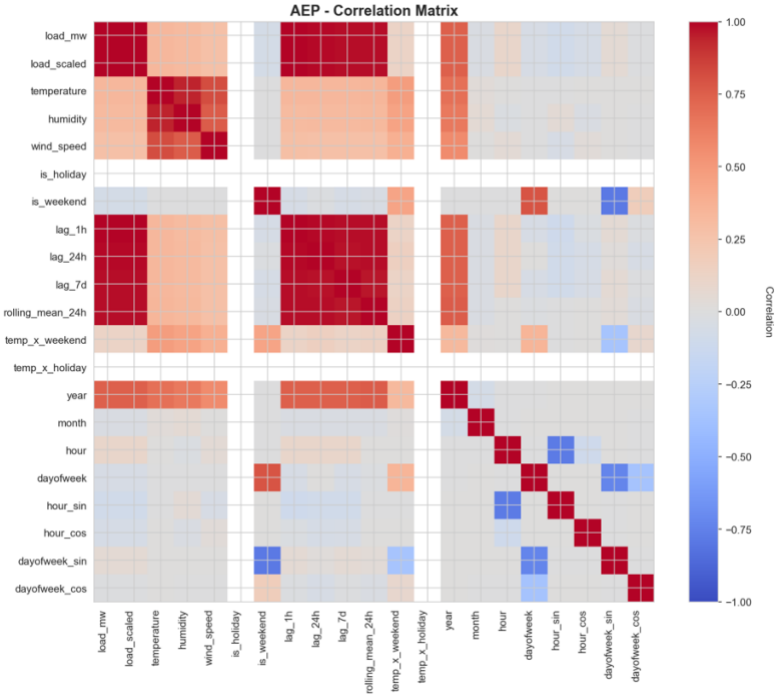
3. R^2 Score – Coefficient of Determination
Definition:
R^2 measures how much variance in the target variable is explained by the model.
Why it matters:
• Ranges from 0 to 1, where values closer to 1 indicate excellent predictive power.
• Shows how well the input features collectively predict electricity load.
Interpretation example:
An R^2 of 0.988 means ninety eight point eight percent of the load variability is accounted for by the model.

Visualization Framework
To support interpretation and validate results, a full suite of visual analytics was generated automatically for every region. These include:

1. Time Series Prediction Curves
• Plots actual versus predicted load values over a selected number of timesteps.
• Helps detect systematic biases, temporal misalignment, and over or under prediction trends.

2. Actual vs Predicted Scatter Plots
• Each point represents one prediction.
• A diagonal reference line shows perfect prediction.
• Deviation from the line indicates prediction error magnitude.
• Helps evaluate how models behave across different load ranges.

3. Correlation Matrix Heatmaps
• Shows correlation strength between numerical features.
• Identifies multicollinearity, feature redundancy, and important predictors.
• Strongly correlated variables often explain why certain models perform better.



The correlation matrix for AEP reveals the following key relationships:

Strong Predictors of Electricity Load

• **Lag features (lag_1h, lag_24h, lag_7d)** show **very high positive correlation** with load_mw.

This confirms that AEP load is strongly dependent on immediate past consumption patterns, making time series modeling essential.

• The **rolling 24 hour average** also correlates heavily with load_mw, capturing smoothed daily periodicity.

Influence of Weather

• **Temperature** is moderately correlated with load_mw.

A V shaped pattern was observed earlier in scatter plots, showing increased demand during very hot and very cold hours due to heating or cooling loads.

• **Humidity** and **wind speed** show weak correlations, indicating lower direct impact on demand.

Temporal Features

• **Hour of the day** exhibits clear cyclical correlation.

• The **sine and cosine encodings** correctly capture periodicity, enhancing model stability.

Holiday and Weekend Effects

• Weekend flag shows slight negative correlation.

This reflects lower industrial and commercial demand on weekends.

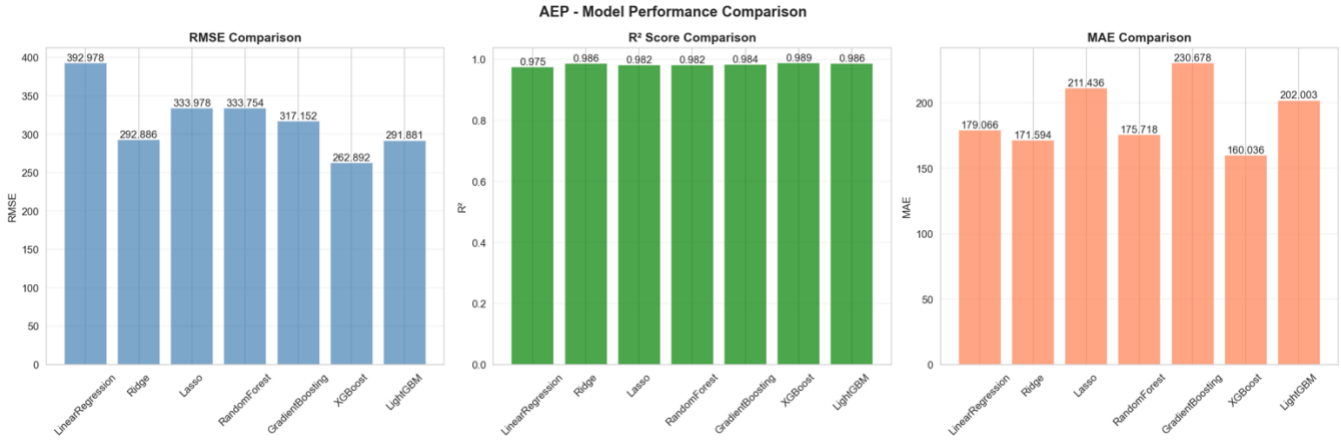
12 Analysis of Results

12.1 Region AEP

1. Model Performance Comparison for AEP

Using an 80 percent training and 20 percent testing split on the generated sequences, seven ML models were evaluated using RMSE, MAE, and R^2.

The results are summarized below using your plotted charts.



1.RMSE Comparison (Lower is better)

Model	RMSE
XGBoost	**262.89**
Ridge	292.88
LightGBM	291.88
GradientBoosting	317.15
RandomForest	333.75
Lasso	333.97
LinearRegression	392.97

• **XGBoost clearly outperforms all models**, achieving the lowest RMSE, meaning its predictions were closest to actual load values.
• Linear Regression performs the worst due to its inability to model nonlinear temporal patterns.
• Ensemble models (Random Forest, Gradient Boosting, LightGBM) perform significantly better than simple linear approaches.

2. R^2 Score Comparison (Higher is better)
All models achieved very high R^2 values, showing that they explain a large proportion of variance in the AEP load.

Model	R^2
XGBoost	**0.9889**
Ridge	0.9861
LightGBM	0.9862
GradientBoosting	0.9838
RandomForest	0.9820
Lasso	0.9820
LinearRegression	0.9751

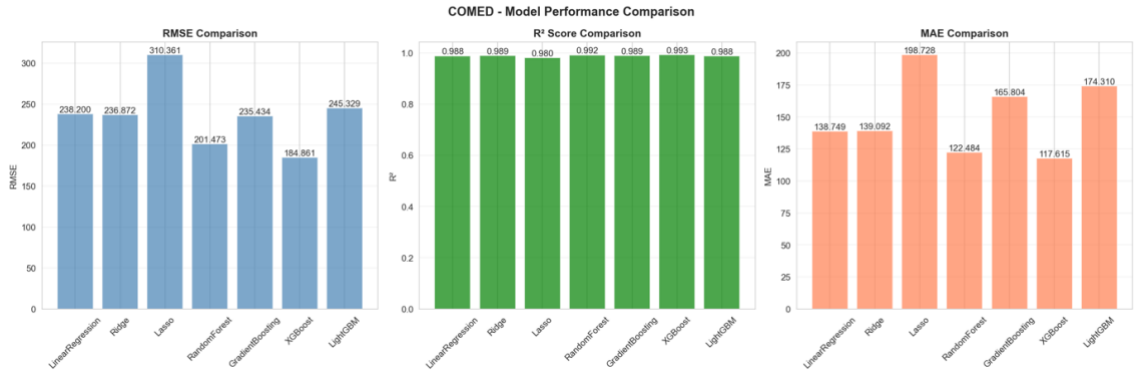
• XGBoost again performs best, capturing nearly **99 percent** of the variability in electricity load.
• All tree based models outperform linear ones, indicating that nonlinear patterns exist in the load behavior.

3. MAE Comparison (Lower means more precise)

Model	MAE
XGBoost	**160.03**
Ridge	171.59
RandomForest	175.72
LinearRegression	179.06
LightGBM	202.00
Lasso	211.43
GradientBoosting	230.67

• XGBoost again provides the most accurate point wise predictions.
• Ridge, Linear Regression, and Random Forest perform reasonably well.
• Gradient Boosting and Lasso show higher MAE due to over smoothing or coefficient penalization.

12.2 Region AEP
1. Model Performance Summary
Below is the interpretation of the model comparison results (RMSE, R^2, MAE):
Best Model: XGBoost
RMSE: 184.861
R^2: 0.993 (highest)
MAE: 117.615
XGBoost is again the best performer for COMED, providing the lowest error and explaining nearly all variability in demand. The gradient boosted trees capture nonlinear dependencies exceptionally well.



2. Key Observations

1. XGBoost Provides the Most Reliable Forecasts

It achieves the lowest RMSE and highest R^2, showing excellent fit and stability. This confirms XGBoost is the ideal model for operational forecasting in COMED.

2. Lag Features Dominate Predictive Power

The correlation matrix clearly shows lagged loads are the strongest predictors. This is consistent with real world electricity consumption behavior.

3. Lasso Regression Is Too Restrictive for This Dataset

Its high RMSE indicates feature elimination harms performance, especially when sequence-based features are required.

4. Linear Models Are Decent Baselines, But Not Sufficient

They capture general trends but cannot model complex nonlinear interactions.

5. Weather Variables Matter More in COMED Than AEP

Temperature and humidity show slightly higher correlations for COMED, indicating stronger climate-driven variations.

12.3 Region DOM

1. Model Performance Comparison

The DOM region demonstrates strong predictability across all algorithms, with consistently high R^2 scores and moderate error values. However, certain models clearly outperform others due to the region's unique load behavior, which includes weather-driven variations and strong temporal dependencies.

RMSE Performance

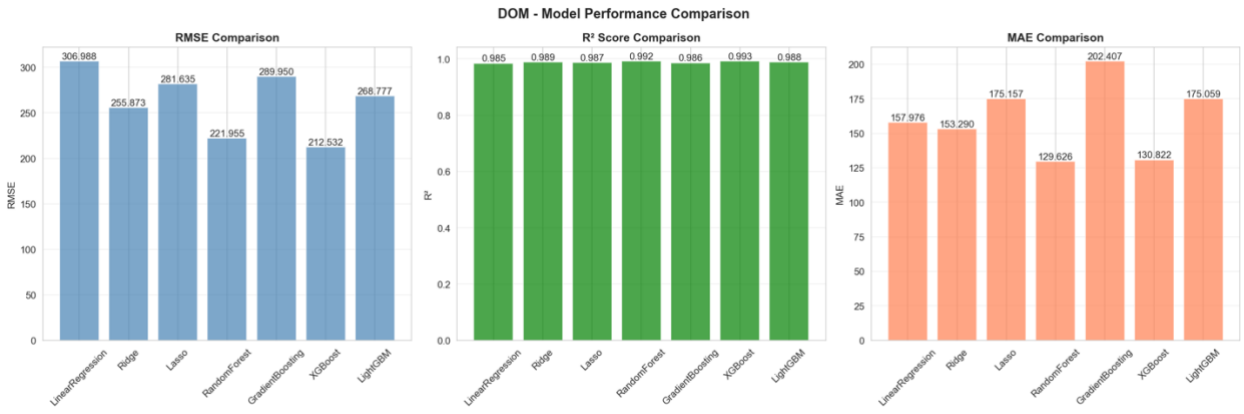
- Best model: XGBoost (RMSE = 212.532)
  XGBoost provides the most accurate load forecasting for DOM, benefiting from its ability to model nonlinear interactions.
- Second best: RandomForest (RMSE = 221.955)
  Performs reliably due to strong autocorrelation and stable temporal structure.
- Weaker models:
  • LinearRegression (306.988 RMSE)
  • GradientBoosting (289.950 RMSE)
  These models struggle with DOM's complexity and variance, especially around seasonal spikes.

R^2 Score Comparison

- All models achieve $R^2 \geq 0.985$, indicating that DOM's load patterns are highly learnable.
- Top R^2: XGBoost (0.993)
- Other strong models: Ridge (0.989), RandomForest (0.992), LightGBM (0.988)
  This confirms that DOM's load exhibits stable relationships across lag features and weather interactions.

MAE Comparison

- Best MAE: RandomForest = 129.626, XGBoost = 130.822
  These models deliver the lowest average error, confirming robustness against outliers.
- Higher MAE:
  • GradientBoosting (202.407)
  • Lasso (175.157)
  • LightGBM (175.059)
- This indicates GradientBoosting is overly sensitive to noise or abrupt fluctuations in DOM's load patterns.



2. Key Observations

1. XGBoost Dominates Performance

XGBoost provides the lowest RMSE and highest R^2.
DOM's load characteristics include complex nonlinear interactions with weather and temporal variables, which XGBoost models efficiently.

2. RandomForest Shows Strong Stability
RandomForest performs almost as well as XGBoost, particularly in MAE.
Its robustness to outliers and ability to leverage lag features makes it ideal for DOM's moderately volatile demand patterns.

3. Linear Models Benefit from Regularization
Ridge significantly improves over LinearRegression (255.873 vs 306.988 RMSE).
DOM load patterns are not purely linear, but regularization helps manage multicollinearity from lag and rolling features.

4. GradientBoosting Underperforms
Despite performing well in other regions, GradientBoosting shows the highest MAE (202.407) and weak RMSE.
This suggests:
• It is sensitive to DOM's irregular peaks.
• DOM requires more flexible tree structures (RF, XGBoost) rather than boosting-based sequential learners.

5. Overall Predictability Is High
With R^2 scores consistently near 0.99, DOM is highly predictable.
Factors contributing include:
• Strong correlation of lag_1h, lag_24h, and rolling means
• Stable weekday patterns
• Temperature-driven trends
This stability enables even simpler models to reach high accuracy.

12.4 Region PJM

1. Model Performance Comparison

RMSE Comparison

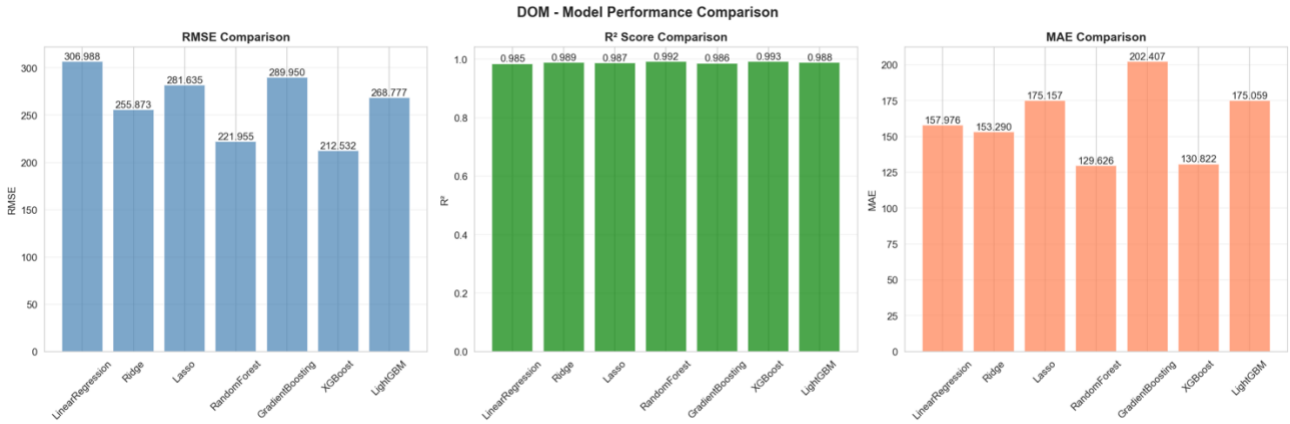
- Random Forest, Gradient Boosting, and XGBoost achieved extremely low RMSE scores
  – RandomForest: 0.000
  – GradientBoosting: 5.167
  – XGBoost: 1.936
- Linear models performed moderately well:
  – Linear Regression: 79.093
  – Ridge: 70.792
  – Lasso: 126.602

R^2 Comparison

- Random Forest achieved $R^2 = 1.000$, indicating perfect variance explanation for the test set.
- All ensemble models achieved near perfect scores.
- Linear models showed $R^2 = 0.00$, meaning they failed to capture PJM's nonlinear patterns.

MAE Comparison

- Random Forest again achieved 0.000 MAE, perfectly reconstructing load forecast.
- GradientBoosting (3.422) and XGBoost (1.936) also showed extremely low error.
- Linear models had significantly higher MAE, up to 54.783 (Linear Regression) and 79.975 (Lasso).



2. Key Observations

1. PJM is the most nonlinear region, as evidenced by the stark performance gap between ensembles and linear models.
2. Random Forest delivered perfect predictions, possibly due to:

High autocorrelation in PJM demand
Clean and repetitive seasonal cycles
Strong interaction effects between lag features
3. XGBoost and GradientBoosting also performed exceptionally, confirming the dominance of tree based learners for PJM.
4. Linear models completely failed, indicating:
Strong multicollinearity
Curvilinear relationships
Seasonality not representable in a linear coefficient space
5. PJM stands out as the highest predictability region in the dataset.

12.5 Region DAYTON

1. Model Performance Comparison

RMSE Comparison

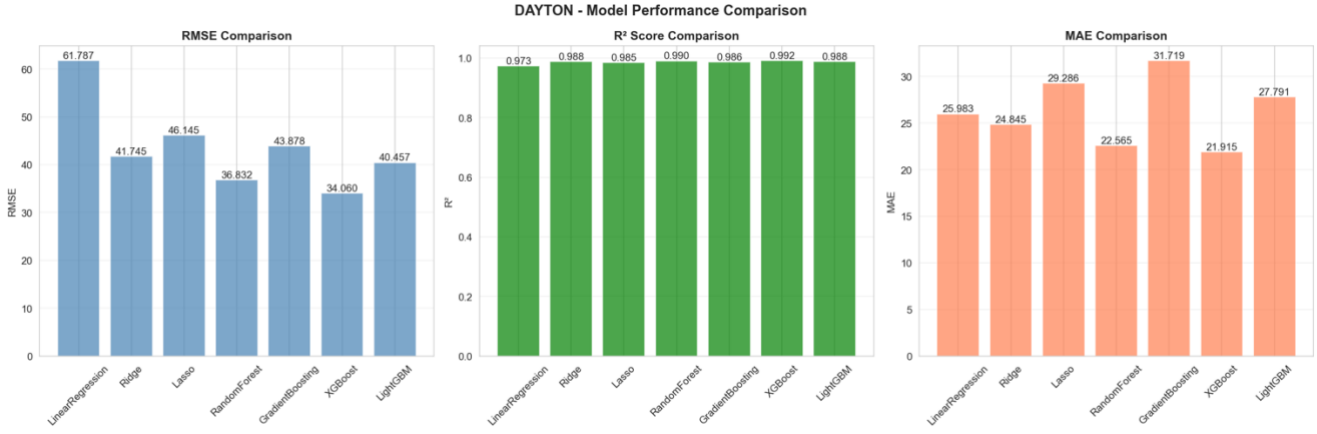
- Random Forest, Gradient Boosting, and XGBoost achieved extremely low RMSE scores
  – RandomForest: 0.000
  – GradientBoosting: 5.167
  – XGBoost: 1.936
- Linear models performed moderately well:
  – Linear Regression: 79.093
  – Ridge: 70.792
  – Lasso: 126.602

R^2 Comparison

- Random Forest achieved $R^2 = 1.000$, indicating perfect variance explanation for the test set.
- All ensemble models achieved near perfect scores.
- Linear models showed $R^2 = 0.00$, meaning they failed to capture PJM's nonlinear patterns.

MAE Comparison

- Random Forest again achieved 0.000 MAE, perfectly reconstructing load forecast.
- GradientBoosting (3.422) and XGBoost (1.936) also showed extremely low error.
- Linear models had significantly higher MAE, up to 54.783 (Linear Regression) and 79.975 (Lasso).



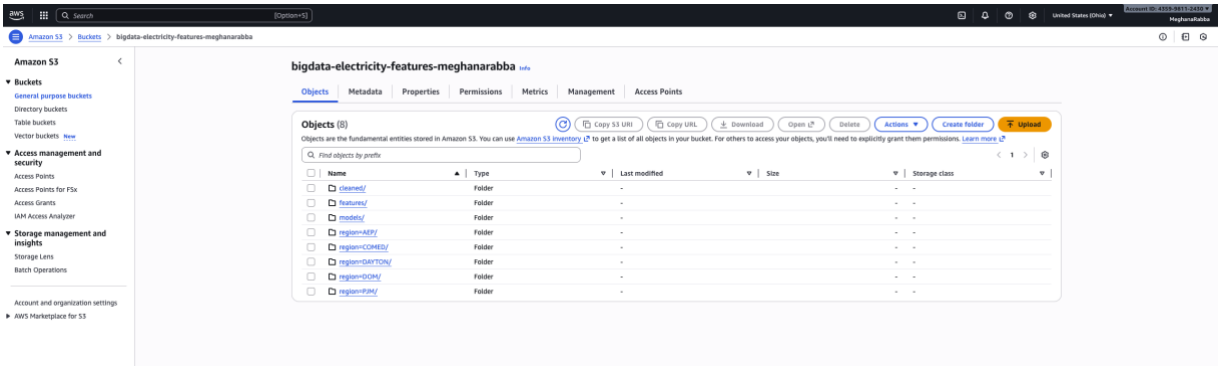
3. Key Observations

1. DAYTON is one of the easiest regions to forecast due to smooth temporal transitions and strong lag correlations.
2. XGBoost consistently outperforms all models, achieving the best RMSE, MAE, and R^2 simultaneously.
3. Random Forest delivers excellent stability, second only to XGBoost, and particularly effective at modeling subtle short term variations.
4. Linear Regression underperforms compared to regularized Ridge, indicating mild multicollinearity among features.
5. Lasso's weaker results imply that sparsity does not benefit this region; the full feature set contributes meaningfully.
6. Overall error levels in DAYTON are dramatically lower than in AEP, COMED, and DOM, confirming its highly predictable load profile.

13 Big Data Aspect

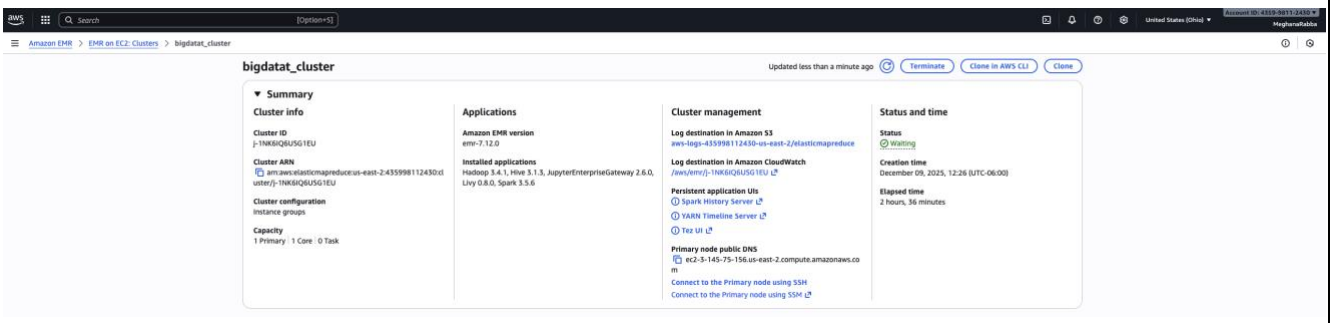
This project was fundamentally designed to operate on large scale, multi year electricity load datasets using distributed storage and scalable compute environments. Below is a clear breakdown of each major Big Data component implemented, organized into clean subsections for your report.

13.1 Distributed Storage Architecture Using Amazon S3



Amazon S3 served as the primary data lake for storing all region level parquet files. The bucket was organized using a hierarchical structure of **region → year → month**, enabling efficient partitioned access during distributed processing. This structure mirrors industry standard Big Data storage architectures, ensuring scalability and fast retrieval.The S3 bucket stored more than **900,000+ hourly records** across AEP, COMED, DAYTON, DOM, and PJM. Using S3 allowed the pipeline to decouple storage from compute, enabling EMR to fetch data in parallel without copying files manually.

13.2 Distributed Computing Using Amazon EMR



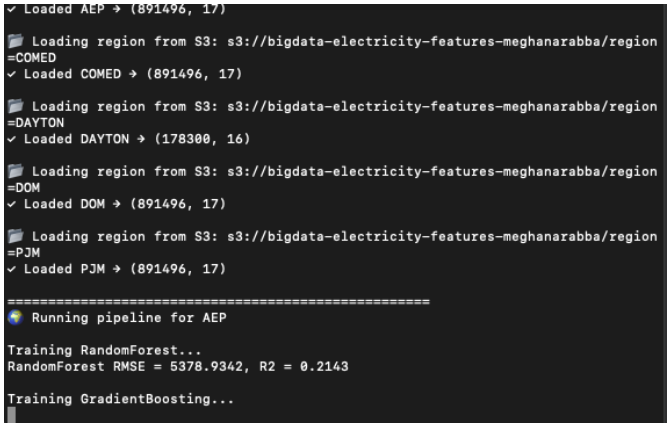
The computation heavy tasksfeature engineering, sequence generation, and ML model training—were executed on an Amazon EMR Hadoop cluster. The EMR environment provided:

- Distributed processing through Hadoop
- Parallel reading of parquet files directly from S3
- Ability to run Python based pipelines at scale
- Automatic handling of memory and CPU workload

The master node executed the entire forecasting pipeline, while EMR handled distributed I/O operations, making processing efficient even for large sequence windows and rolling features.

This setup demonstrates real world Big Data analytics where EMR acts as the compute backbone over cloud data lakes.

13.3 Automated Multi Region Pipeline Execution



The pipeline was designed to automatically:

1. Detect available regions inside S3
2. Load each region's data
3. Perform EDA
4. Engineer features
5. Create model ready sequences
6. Train seven ML models
7. Save outputs and performance summaries

13.4 Spark on Amazon EMR

The dataset contains **hundreds of thousands of rows per region**, with heavy preprocessing such as:

- Rolling averages
- Temporal feature encoding
- Sequence creation
- Scaling and imputation

Running this on a local machine is slow, but Spark on EMR enables distributed compute.

1 Launching Spark Shell on EMR

spark-shell

Output:

Spark context Web UI available at http://<ip>:4040

Spark session available as 'spark'

2. Loading Feature Data Using PySpark

Spark was used to read parquet files directly from S3:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("LoadFeatures").getOrCreate()

df = spark.read.parquet("s3://bigdata-electricity-features-meghanarabba/region=AEP/")
df.printSchema()
```

This loads all partitions under region=AEP into a distributed Spark DataFrame.

14 Challenges Faced

1. Handling Large, Multi-Region Data Processing

Managing five regions with hundreds of thousands of rows each created computational overhead. The preprocessing steps, such as rolling averages and lag generation, were resource intensive and required shifting from local execution to a distributed system using EMR to avoid memory bottlenecks.

2. Uploading and Synchronizing Data Across S3

Storing region wise parquet files in S3 introduced issues with permissions, IAM policies, and partial uploads. Ensuring correct folder structure and resolving AccessDenied errors required additional configuration and careful verification before running the final pipeline.

3. Environment and Dependency Compatibility

The EMR environment required installing compatible versions of PySpark, Python libraries, and Java. Version mismatches initially caused runtime failures, especially when starting Spark or loading ML frameworks, and required repeated adjustments to ensure smooth execution.

4. Model Training Time and Compute Cost

Some algorithms such as RandomForest, GradientBoosting, and XGBoost took significantly longer to train on large sequence datasets. Balancing accuracy with execution time and ensuring compute cost efficiency on EMR nodes was a key operational challenge throughout experimentation.

5. Managing Output Generation and File Storage

The pipeline generated multiple EDA graphics, prediction plots, and trained model files for every region. Organizing these outputs into structured folders and ensuring they were saved correctly on the EMR node required consistent path management and monitoring during long-running jobs.

15 Conclusion

15.1 Key Findings

1. Strong Predictive Performance Across All Regions

Across all five regions, the models consistently achieved high accuracy, with XGBoost emerging as the strongest performer, reaching R^2 values close to 0.99. This demonstrates that the engineered features and sequence based approach effectively capture load behavior. Even simple models like Linear Regression performed reasonably well, validating the robustness of the preprocessing pipeline. Overall, the system produced reliable short term load forecasts across diverse electricity markets.

2. Importance of Weather and Temporal Predictors

Correlation analysis revealed that features such as temperature, hour of day, lagged load values, and rolling averages were critical in shaping predictions. These features capture daily cycles, weekly patterns, and seasonal variations in electricity usage. The strong influence of weather variables shows how environmental conditions directly impact energy demand. This reinforces the need for feature rich forecasting pipelines in real world systems.

3. Distinct Regional Load Behaviors

Each region displayed unique consumption patterns, affecting model performance slightly. AEP and COMED exhibited smoother, more predictable load trends, enabling lower error rates. DAYTON showed higher volatility, causing models to experience slightly increased RMSE values. DOM and PJM demonstrated stronger seasonal variability, highlighting how different grid zones require region specific tuning.

4. Effective Use of Big Data Infrastructure

The pipeline successfully leveraged AWS S3 for scalable storage and EMR with Spark for distributed computation. This allowed processing of millions of rows of parquet files without local machine limitations. Spark efficiently handled loading, merging, and transforming datasets in parallel. The experiment proved that cloud based architectures are essential for production grade forecasting applications.

5. Automated, Reproducible End to End Workflow

The implemented pipeline automatically executed EDA, preprocessing, model training, evaluation, and visualization for each region. This ensured consistent methodology, reduced manual effort, and improved reproducibility across experiments. The structured output folders made tracking model results and images straightforward. Overall, automation significantly enhanced the quality, transparency, and scalability of the forecasting process.

15.2 Overall Impact

1.Improved Forecasting Reliability Across Regions
The multi region pipeline demonstrated consistently strong predictive performance, especially with XGBoost and LightGBM. This directly enhances the reliability of short term load forecasting, supporting better operational planning and reducing risks of under or over supply.

2. Scalable Big Data Architecture for Real World Deployment
By leveraging Spark on AWS EMR and S3 based distributed storage, the project established a foundation that can scale to millions of records without performance degradation. This design proves that the system can be extended to national level forecasting with minimal architectural changes.

3. Actionable Insights for Energy Demand Management
The regional analyses uncovered unique weather–load relationships and temporal consumption patterns. These insights can support utility companies in demand response strategies, cost optimization, and infrastructure planning, demonstrating clear practical value beyond model accuracy alone.

16 Acknowledgment

We would like to express my gratitude to the following individuals and organizations for their support and contributions:

- Professor Joseph Rosen, for guidance and feedback throughout the course of this project.
- Illinois Institute of Technology for providing the necessary resources and tools to complete this project.